

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 3**



LINKED LIST

Oleh:

Bi'ahlil Haq Aulia Akbar Awaludin

NIM. 2010817110011

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI
2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 3

Laporan Praktikum Algoritma & Struktur Data Modul 3: Linked List ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Bi'ahlil Haq Aulia Akbar Awaludin
NIM : 2010817110011

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN.....	i
DAFTAR ISI.....	ii
DAFTAR GAMBAR.....	iii
DAFTAR TABEL.....	iv
SOAL 1.....	1
A. Source Code.....	1
B. Pembahasan.....	5
SOAL 2.....	7
A. Source Code.....	8
B. Output Program.....	9
C. Pembahasan.....	9
SOAL 3.....	11
A. Source Code.....	11
B. Pembahasan.....	15
SOAL 4.....	17
A. Source Code.....	18
B. Output Program.....	20
C. Pembahasan.....	20
TAUTAN GIT.....	22

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 2.....	9
Gambar 2. Screenshot Hasil Jawaban Soal 2.....	20

DAFTAR TABEL

<i>Table 1. Source Code single_linked_list.cpp.....</i>	<i>5</i>
<i>Table 2. Contoh input dan output soal 2.....</i>	<i>8</i>
<i>Table 3. Source Code prob_a.cpp.....</i>	<i>9</i>
<i>Table 4. Source Code double_linked_list.cpp.....</i>	<i>15</i>
<i>Table 5. Contoh input dan output soal 4.....</i>	<i>18</i>
<i>Table 6. Source Code prob_b.cpp.....</i>	<i>20</i>

SOAL 1

Implementasi Senarai Berantai Tunggal Melingkar

Pada file `single_linked_list.h` yang diberikan, implementasikan struktur `SingleLinkedList` beserta fungsi-fungsinya ke dalam file `single_linked_list.cpp`.

Jelaskan alur algoritma dari setiap metode yang diimplementasikan. Pastikan metode `clear()` berjalan dengan benar untuk menghindari memory leak (kebocoran memori). Lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi menggunakan Big-O Notation. Seluruh manajemen memori harus dialokasikan secara manual di dalam heap.

Jelaskan menggunakan alur bagaimana dan mengapa. Jelaskan kenapa algoritma yang kalian rancang itu valid dan yakin benar.

A. Source Code

`single_linked_list.cpp`

```
1  #include "single_linked_list.h"
2  #include <iostream>
3  #include <stdexcept>
4
5  void SingleLinkedList::init() {
6      head = nullptr;
7      tail = nullptr;
8      size = 0;
9  }
10
11 bool SingleLinkedList::is_empty() { return size == 0; }
12
13 void SingleLinkedList::add_front(int val) {
14     Node *new_node = new Node();
15     new_node->data = val;
16
17     if (is_empty()) {
18         new_node->next = new_node;
19         head = new_node;
20         tail = new_node;
21     } else {
22         new_node->next = tail->next;
23         tail->next = new_node;
24         head = new_node;
25     }
```

```

26     size++;
27 }
28
29 void SingleLinkedList::add_back(int val) {
30     Node *new_node = new Node();
31     new_node->data = val;
32
33     if (is_empty()) {
34         new_node->next = new_node;
35         head = new_node;
36         tail = new_node;
37     } else {
38         new_node->next = tail->next;
39         tail->next = new_node;
40         tail = new_node;
41     }
42     size++;
43 }
44
45 void SingleLinkedList::add_idx(int val, int idx) {
46     if (idx < 0 || idx > size) {
47         // size pake > karena zero based index size 99 posisi
48         // nya 98
49         // dan yang diminta disini jika param idx artinya
50         // posisi idx bukan idx - 1
51         // misal idx 5 berarti diminta posisi 5 (0,1,2,3,4,5)
52         // bukan 4
53         throw std::out_of_range("Index node melebihi size
54 saat ini");
55     } else if (idx == 0) {
56         add_front(val);
57     } else if (idx == size) {
58         add_back(val);
59     } else {
60         Node *new_node = new Node();
61         new_node->data = val;
62
63         Node *current = head;
64         for (int i = 0; i < idx - 1; i++) {
65             current = current->next;
66         }
67         new_node->next = current->next;
68         current->next = new_node;
69         size++;

```

```

70     }
71 }
72
73 void SingleLinkedList::delete_front() {
74     if (is_empty()) {
75         throw std::logic_error("List Kosong");
76     } else if (size == 1) {
77         delete head;
78         head = nullptr;
79         tail = nullptr;
80     } else {
81         Node *old_head = head;
82         head = head->next;
83         tail->next = head;
84         delete old_head;
85     }
86     size--;
87 }
88
89 void SingleLinkedList::delete_back() {
90     if (is_empty()) {
91         throw std::logic_error("List Kosong");
92     }
93     if (size == 1) {
94         delete head;
95         head = nullptr;
96         tail = nullptr;
97     } else {
98         Node *current = head;
99         while (current->next != tail) {
100             current = current->next;
101         }
102         current->next = head;
103         delete tail;
104         tail = current;
105     }
106     size--;
107 }
108
109 void SingleLinkedList::delete_idx(int idx) {
110     if (is_empty()) {
111         throw std::logic_error("List Kosong");
112     } else if (idx < 0 || idx >= size) { // size pake >=
113         karena zero based index

```



```

114 // size 99 posisi
115 nya 98
116     throw std::out_of_range("Index node melebihi size
117 saat ini");
118     } else if (idx == 0) {
119         delete_front();
120     } else if (idx == size - 1) {
121         delete_back();
122     } else {
123         Node *current = head;
124         for (int i = 0; i < idx - 1; i++) {
125             current = current->next;
126         }
127         Node *to_delete = current->next;
128         current->next = to_delete->next;
129         delete to_delete;
130         size--;
131     }
132 }
133
134 void SingleLinkedList::display() {
135     if (is_empty()) {
136         std::cout << "List Kosong\n";
137     } else {
138         Node *current = head;
139         for (int i = 0; i < size; i++) {
140             std::cout << current->data << " -> ";
141             current = current->next;
142         }
143         std::cout << "Kembali Ke Head (Head Data : " << head-
144 >data << ")\n";
145     }
146 }
147
148 int SingleLinkedList::get(int idx) {
149     if (is_empty()) {
150         throw std::logic_error("List Kosong");
151     } else if (idx < 0 || idx >= size) {
152         throw std::out_of_range("Index node melebihi size
153 saat ini");
154     } else {
155         Node *current = head;
156         for (int i = 0; i < idx; i++) {
157             current = current->next;

```

```

158     }
159     return current->data;
160 }
161 }
162
163 void SingleLinkedList::set(int val, int idx) {
164     if (is_empty()) {
165         throw std::logic_error("List Kosong");
166     } else if (idx < 0 || idx >= size) {
167         throw std::out_of_range("Index node melebihi size
168 saat ini");
169     } else {
170         Node *current = head;
171         for (int i = 0; i < idx; i++) {
172             current = current->next;
173         }
174         current->data = val;
175     }
176 }
177
178 void SingleLinkedList::clear() {
179     while (!is_empty()) {
180         delete_front();
181     }
182 }

```

Table 1. Source Code single_linked_list.cpp

B. Pembahasan

Alur Fungsi Utama

- Penambahan di ujung (add_front dan add_back): Pertama memori dinamis akan dialokasikan pada heap untuk node baru. Node baru kemudian diisi dengan nilai data. Kemudian cek jika list masih kosong maka node akan menunjuk dirinya sendiri dan menjadi head sekaligus tail. Jika tidak kosong penunjuk next dari node baru dibuat menunjuk head lama, kemudian tail → next menunjuk node baru. Disini perbedaan pada add_front dan add_back, untuk front head dipindah ke node baru sedangkan back tail yang dipindah ke node baru.
- Penambahan node di posisi yang bebas: program disini memvalidasi batas index, jika valid pointer bantuan akan berjalan (traverse) mulai dari head dan berhenti di node index – 1. kemudian node baru next akan menunjuk node sebelumnya, lalu node sebelum mengubah next nya untuk menunjuk node baru.

- Penghapusan node (delete_front, delete_back, delete_idx):
 - front: head digeser ke node didepannya, kemudian tail → next diubah menunjuk ke head baru. Node head lama dihapus (delete).
 - Back dan idx: buat pointer bantuan, pointer ini akan berjalan ke node sebelum target index yang ingin dihapus. Next dari node sebelum target dialihkan agar melewati target menunjuk node setelah target. Setelah list aman node target akan dihancurkan dari memori.
- Pembersih memori / list (clear): fungsi ini melakukan perulangan while dengan kondisi !is_empty() yang memanggil delete_front() secara terus menerus. Karena fungsi penghapus sudah akan memanggil delete, dan menghapus list.

Kenapa Ini Bekerja?

Kenapa add_front dan add_back mirip bedanya hanya pada penempatan head dan tail. Hal ini karena struktur linked list yang melingkar, penambahan di depan atau belakang menempati titik yang sama yaitu antara head dan tail.

Karena setiap fungsi delete disini sudah memanggil delete diakhir maka ini sudah mencegah memory leak dan karena itu clear hanya perlu memanggil fungsi delete saja untuk mengosongkan list.

Pada add atau delete di idx kenapa kita menggunakan validasi yang sedikit berbeda? Karena untuk front kita bisa memasukan node pada posisi $idx > size$ karena ini zero based jadi jika idx 4 itu size nya 5 dan kita ingin masukan di posisi/idx 5 maka kita hanya perlu panggil add_back, berbeda dengan delete idx kita memastikan posisi $idx \geq size$ disini mengisaratkan bahwa kita tidak bisa menghapus idx 5 jika size cuman 5 karena size lima posisi paling akhir adalah 4.

Analisis Kompleksitas

Kompleksitas waktu: $O(1)$ untuk add_front, add_back, delete_front. Tetapi $O(n)$ untuk add_idx, delete_idx, get, dan set karena fungsi-fungsi ini memerlukan traversal (looping) dari head ke indeks tujuan (n).

Kompleksitas Ruang: $O(1)$ memori untuk tiap fungsi nya, tetapi karena ini adalah struktur data maka secara keseluruhan Struktur data linked list ini memiliki $O(n)$ kompleksitas.

SOAL 2

Si Palui dan Relikui Siklus Resonansi

Si Palui terjebak di dalam sebuah kuil kuno yang dikelilingi oleh N buah batu relikui yang membentuk formasi lingkaran sempurna. Untuk membuka pintu keluar, ia harus menghancurkan batu-batu tersebut dengan urutan tertentu menggunakan pancaran energi beruntun.

Pancaran energi bermula dari batu pertama, lalu melompat sejauh K langkah ke arah depan, lalu menghancurkan batu target tersebut. Namun, kuil ini memiliki anomali energi dinamis:

- Jika nilai ukiran pada batu yang baru saja dihancurkan adalah bilangan genap, maka lompatan berikutnya akan bertambah jauh ($K=K+1$).
- Jika nilai ukiran pada batu yang dihancurkan adalah bilangan ganjil, maka lompatan berikutnya akan melemah ($K=K-1$).
- Jika pada suatu titik energi lompatan habis atau negatif ($K\leq 0$), sistem akan mengalami reset dan mengembalikan nilai lompatan ke nilai K awal (saat pancaran energi pertama kali diinisialisasi).

Setelah seluruh batu hancur kecuali satu, batu terakhir itulah yang akan menjadi kunci pintu keluar.

Batasan

- $1 \leq N \leq 104$
- $1 \leq K \leq 109$
- $1 \leq A_i \leq 106$ (Nilai ukiran pada batu)
- Program wajib menggunakan implementasi struktur data yang dibuat dari Soal 1.

Format Input

N K

A_1 A_2 ... A_N

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang simbol
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Sebuah bilangan bulat tunggal yang merupakan nilai ukiran pada batu terakhir yang tersisa.

Input	Output
5 2 1 4 3 6 2	6
4 5 10 11 12 13	12

Table 2. Contoh input dan output soal 2

A. Source Code

prob_a.cpp

```

1 #include "single_linked_list.h"
2 #include <iostream>
3 using namespace std;
4
5 int main() {
6     // Input n k
7     int N;
8     long long K; // long long karena K bisa sampai 10^9
9     cin >> N >> K;
10    long long initial_K = K;
11    // init
12    SingleLinkedList relikui;
13    relikui.init();
14    // Masukkan semua batu ke dalam list
15    for (int i = 0; i < N; i++) {
16        int batu;
17        cin >> batu;
18        relikui.add_back(batu);
19    }
20    // Mulai dari batu pertama
21    long long pos = 0;
22    // Loop selama batu yang tersisa lebih dari 1
23    while (relikui.size > 1) {
24        // Hitung lompatan
25        long long lompatan_asli = (K - 1) % relikui.size;
26        pos = (pos + lompatan_asli) % relikui.size;
27        // Ambil nilai ukiran batu tersebut
28        int nilai_batu = relikui.get(pos);
29        // Hancurkan batunya!
30        relikui.delete_idx(pos);
31        // batu "selanjutnya" otomatis menempati indeks 'pos'
32        yang sekarang!
33        pos = pos % relikui.size;
34        // cek anomali

```

```

35     if (nilai_batu % 2 == 0) {
36         K = K + 1;
37     } else {
38         K = K - 1;
39     }
40     // Aturan Reset K
41     if (K <= 0) {
42         K = initial_K;
43     }
44 }
45 // Cetak 1 batu yang tersisa
46 cout << relikui.get(0) << endl;
47 return 0;
48 }

```

Table 3. Source Code prob_a.cpp

B. Output Program

```

kimsora .../linked-list ? main [?] v15.2.0
> ./prob_a
5 2
1 4 3 6 2
6

kimsora .../linked-list ? main [?] v15.2.03s
> ./prob_a
4 5
10 11 12 13
12

kimsora .../linked-list ? main [?] v15.2.04s
> ./prob_a
7 57
492 351 542 521 877 544 463
351

```

Gambar 1. Screenshot Hasil Jawaban Soal 2

C. Pembahasan

File ini adalah main file dimana program utama berjalan.

Alur Algoritma

- Inisialisasi data: memasukan semua angka uiran batu secara berurutan ke bagian belakang list dengan fungsi `add_back`.
- Loop Eksekusi: Loop `while` yang terus berjalan selama jumlah batu di dimensi masih lebih dari 1.
- Kalkulasi lompatan: menghitung jarak lompatan, untuk menghindari infinite loop saat angka K yang sangat besar, kita menggunakan matematika modulo. Posisi batu target dicari dengan rumus $pos = (pos + K + 1) \% size$.
- Penghancuran: nilai batu pada posisi target dibaca lalu batu tersebut dihancurkan dengan `delete_idx`. Setelah batu hancur batu setelahnya otomatis bergeser merapat ke kiri sendiri.
- Anomali Energi: kita melakukan cek nilai batu yang akan dihancurkan. Jika genap nilai lompatan K ditambah 1. jika ganjil lompatan K dikurangi 1. Jika K turun hingga 0 atau negatif K akan koreset ke nilai awalnya.
- Pencetak: ketika `while` loop selesai program akan memanggil fungsi `get(0)` untuk mencetak hasil akhir.

Kenapa Ini Bekerja?

Algoritma Bekerja bagus karena menerapkan sifat melingkar (Circular) dari posisi formasi batu yang sebenarnya. Penggunaan rumus Modulo ($\% size$) pada langkah ke-3 adalah inti dari efisiensi dan keamanan program ini. Sesuai dengan batasan masalah (Constraint), nilai awalan K dapat mencapai skala satu miliar (10^9). Jika hanya melakukan pointer traversal (berjalan satu per satu menggunakan `current = current → next`) sebanyak K yang besar akan memicu kesalahan Constraint Time Limit Exceeded (TLE). Modulo menjadi solusi yang tepat untuk mendapatkan titik koordinat batu secara instan tanpa harus melakukan pointer traversal yang berlebihan. Kemudian `delete_idx` yang digunakan sebagai penghapus menggunakan delete untuk menjaga batas memori Constraint.

Analisis Kompleksitas

Kompleksitas Waktu: $O(n^2)$, karena terdapat loop utama sebanyak N kali untuk menghancurkan batu (`while`). Dan dilama nya `delete_idx` dipanggil yang dimana dia akan traverse sebanyak `idx`. Yang akhir nya akan menjadi $O(n^2)$.

Kompleksitas Ruang: $O(n)$, karena menyimpan N buah batu di dalam single linked list.

SOAL 3

Implementasi Senarai Berantai Ganda Melingkar

Pada file `double_linked_list.h` yang diberikan, implementasikan struktur `DoubleLinkedList` beserta fungsi-fungsinya ke dalam file `double_linked_list.cpp`.

Jelaskan alur algoritma dari setiap metode yang diimplementasikan. Pastikan metode `clear()` berjalan dengan benar untuk menghindari memory leak (kebocoran memori). Lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi menggunakan Big-O Notation. Seluruh manajemen memori harus dialokasikan secara manual di dalam heap.

Jelaskan menggunakan alur bagaimana dan mengapa. Jelaskan kenapa algoritma yang kalian rancang itu valid dan yakin benar.

A. Source Code

`double_linked_list.cpp`

```
1  #include "double_linked_list.h"
2  #include <iostream>
3  #include <stdexcept>
4
5  void DoubleLinkedList::init() {
6      head = nullptr;
7      tail = nullptr;
8      size = 0;
9  }
10
11 bool DoubleLinkedList::is_empty() { return size == 0; }
12
13 void DoubleLinkedList::add_front(char val) {
14     Node *new_node = new Node();
15     new_node->data = val;
16     if (is_empty()) {
17         new_node->next = new_node->prev = new_node;
18         head = new_node;
19         tail = new_node;
20     } else {
21         new_node->next = head;
22         new_node->prev = tail;
23         head->prev = new_node;
24         tail->next = new_node;
25         head = new_node;
26     }
27     size++;
```



```

28 }
30 void DoubleLinkedList::add_back(char val) {
31     Node *new_node = new Node();
32     new_node->data = val;
33     if (is_empty()) {
34         new_node->next = new_node->prev = new_node;
35         head = new_node;
36         tail = new_node;
37     } else {
38         new_node->next = head;
39         new_node->prev = tail;
40         tail->next = new_node;
41         head->prev = new_node;
42         tail = new_node;
43     }
44     size++;
45 }
46 void DoubleLinkedList::add_idx(char val, int idx) {
47     if (idx < 0 || idx > size) {
48         // size pake > karena zero based index size 99 posisi
49         // nya 98
50         // dan yang diminta disini jika param idx artinya
51         // posisi idx bukan idx - 1
52         // misal idx 5 berarti diminta posisi 5 (0,1,2,3,4,5)
53         // bukan 4
54         throw std::out_of_range("Index node melebihi size
55 saat ini");
56     } else if (idx == 0) {
57         add_front(val);
58     } else if (idx == size) {
59         add_back(val);
60     } else {
61         Node *new_node = new Node();
62         new_node->data = val;
63         Node *current = head;
64         for (int i = 0; i < idx - 1; i++) {
65             current = current->next;
66         }
67         new_node->next = current->next;
68         new_node->prev = current;
69         current->next->prev = new_node;
70         current->next = new_node;
71         size++;
72     }

```

```

73 }
74
75 void DoubleLinkedList::delete_front() {
76     if (is_empty()) {
77         throw std::logic_error("List Kosong");
78     } else if (size == 1) {
79         delete head;
80         head = nullptr;
81         tail = nullptr;
82     } else {
83         Node *old_head = head;
84         tail->next = head->next;
85         head->next->prev = tail;
86         head = head->next;
87         delete old_head;
88     }
89     size--;
90 }
91 void DoubleLinkedList::delete_back() {
92     if (is_empty()) {
93         throw std::logic_error("List Kosong");
94     }
95     if (size == 1) {
96         delete head;
97         head = nullptr;
98         tail = nullptr;
99     } else {
100         Node *old_tail = tail;
101         head->prev = tail->prev;
102         tail->prev->next = head;
103         tail = tail->prev;
104         delete old_tail;
105     }
106     size--;
107 }
108 void DoubleLinkedList::delete_idx(int idx) {
109     if (is_empty()) {
110         throw std::logic_error("List Kosong");
111     } else if (idx < 0 || idx >= size) { // size pake >=
112         karena zero based index // size 99 posisi
113                                     // nya 98
114         throw std::out_of_range("Index node melebihi size
115 saat ini");
116

```

```

117     } else if (idx == 0) {
118         delete_front();
119     } else if (idx == size - 1) {
120         delete_back();
121     } else {
122         Node *current = head;
123         for (int i = 0; i < idx; i++) {
124             current = current->next;
125         }
126         current->prev->next = current->next;
127         current->next->prev = current->prev;
128         delete current;
129         size--;
130     }
131 }
132
133 void DoubleLinkedList::display() {
134     if (is_empty()) {
135         std::cout << "EMPTY\n";
136     } else {
137         Node *current = head;
138         for (int i = 0; i < size; i++) {
139             std::cout << current->data;
140             current = current->next;
141         }
142         std::cout << "\n";
143     }
144 }
145 char DoubleLinkedList::get(int idx) {
146     if (is_empty()) {
147         throw std::logic_error("List Kosong");
148     } else if (idx < 0 || idx >= size) {
149         throw std::out_of_range("Index node melebihi size
150 saat ini");
151     } else {
152         Node *current = head;
153         for (int i = 0; i < idx; i++) {
154             current = current->next;
155         }
156         return current->data;
157     }
158 }
159 void DoubleLinkedList::set(char val, int idx) {
160     if (is_empty()) {

```

```

161     throw std::logic_error("List Kosong");
162 } else if (idx < 0 || idx >= size) {
163     throw std::out_of_range("Index node melebihi size
164 saat ini");
165 } else {
166     Node *current = head;
167     for (int i = 0; i < idx; i++) {
168         current = current->next;
169     }
170     current->data = val;
171 }
172 }
173
174 void DoubleLinkedList::clear() {
175     while (!is_empty()) {
176         delete_front();
177     }
178 }

```

Table 4. Source Code double_linked_list.cpp

B. Pembahasan

Pada soal ini ada 7 fungsi, dimana 1 fungsi constructor, 4 getter, dan 2 fungsi utama.

Alur Fungsi Utama

- Penambahan di ujung (add_front dan add_back): sama dengan yang circular single linked list, memori dialokasikan untuk node baru. Namun sekarang ada 2 petunjuk arah (next,prev) untuk setiap node. Node baru diposisikan antara tail dan head. Node baru next menunjuk head dan node baru prev menunjuk tail. Kemudian head prev menunjuk node baru dan tail next menunjuk node baru juga. Sama dengan single perbedaan front dan back disini hanya pada proses akhir yang sama.
- Penambahan tengah list (add_idx): pointer berjalan ke lokasi idx. Setelah sampai node baru mengikat node setelahnya (next), dan juga node sebelumnya (prev). Setelah itu, penunjuk dari node setelahnya (current next prev) dan node sebelumnya (current next) diganti agar menunjuk node baru.
- Penghapus di ujung (delete_front dan back): untuk front cukup simple kita cukup membuat node bantuan untuk menyimpan head dan mengubah ikatan head next prev untuk menunjuk tail, dan tail next menunjuk head next. Setelah itu head menjadi head next, baru node bantuan yang menyimpan head lama dihapus. Untuk back nya sama saja cuman beda nya ya ini tail yang dihapus.

- Penghapusan di tengah (`delete_idx`): karena adanya ikatan mundur (`prev`), kita bisa langsung saja ke index tujuan tidak perlu berhenti di `idx - 1`. setelah itu node sebelah kiri target (`current prev`) dan node kanan (`current next`) dibuat untuk saling mengikat secara langsung dan tidak melalui node target. Setelah node dikeluarkan dari list node bisa dihancurkan.

Kenapa Ini Bekerja?

Pada `delete_idx` kita perlu untuk membuat node yang bersebelahan dari target untuk saling berkaitan dulu untuk mengeluarkan target dari list, baru node target bisa di delete, jika delete dilakukan deluan maka list akan mengalami dangling pointer dan program bisa crash.

Dengan adanya dua arah pointer yang menjadi keunggulan utama struktur ini bisa dirasakan pada `delete_idx` bagian setelah node target ditemukan koneksi antara node tetangga kiri dan kanan langsung bisa dibuat karena akses langsung ke `current prev` dan `current next`. Ini mengurangi pointer temporary yang dibutuhkan dan mengurangi kesalahan logika.

Analisis Kompleksitas

Semua fungsi yang ada pada stack ini memiliki kompleksitas ruang dan waktu $O(1)$, karena tidak ada loop atau pemanggilan fungsi yang recursive berulang sebanyak n dan hanya operasi standar tanpa ada struct yang berpotensi memiliki size n .

SOAL 4

Si Palui dan Tarik Tambang Dimensi

Si Palui mengamati fenomena aneh di atas sebuah gelanggang melingkar yang berisi N karakter. Terdapat dua proyektor anomali, Alpha (dimulai dari head, bergerak maju menggunakan next) dan Beta (dimulai dari tail, bergerak mundur menggunakan prev).

Palui melakukan observasi selama R ronde. Pada setiap ronde:

1. Proyektor Alpha dipaksa melompat maju sebanyak X langkah. Proyektor Beta dipaksa melompat mundur sebanyak Y langkah (simultan).
2. Tabrakan Singular: Jika Alpha dan Beta mendarat di titik atau karakter yang sama persis secara bersamaan, karakter tersebut akan hancur dan lenyap dari lingkaran dimensi. Jika ini terjadi, Alpha berpindah ke karakter next dari karakter yang hancur, dan Beta berpindah ke karakter prev dari karakter yang hancur.
3. Resonansi Bersebelahan (Adjacent): Jika Alpha dan Beta saling bersebelahan langsung, sifat kuantum dari kedua karakter tersebut saling bertukar (data antara kedua node tersebut di-swap).

Tentukan sisa karakter pada lingkaran dimensi setelah R ronde selesai dieksekusi.

Batasan

- $1 \leq N \leq 5000$
- $1 \leq R \leq 5000$
- $1 \leq X, Y \leq 10^{18}$
- Struktur wajib menggunakan implementasi struktur data dari Soal 3.

Format Input

N R
C1 C2 ... CN
X1 Y1
X2 Y2
...

XR YR

Keterangan: C_i adalah satu karakter char tunggal.

Format Output

Karakter yang tersisa, dicetak berurutan mulai dari head hingga ke elemen terakhir, tanpa spasi. Jika dimensi menjadi kosong, cetak EMPTY.

Input	Output
4 2 A B C D 1 1 2 3	ACB
5 3 V W X Y Z 100000 100000 2 2 5 5	VWYZ

Table 5. Contoh input dan output soal 4

A. Source Code

prob_b.cpp

```

1 #include "double_linked_list.h"
2 #include <iostream>
3 using namespace std;
4
5 int main() {
6     // Input N R
7     int N, R;
8     cin >> N >> R;
9     // Init dll Tambang
10    DoubleLinkedList tambang;
11    tambang.init();
12    // Loop input node char
13    for (int i = 0; i < N; i++) {
14        char c;
15        cin >> c;
16        tambang.add_back(c);
17    }
18    // Penandaan bahwa alpha awalnya head dan beta awalnya
19    tail
20    long long pos_alpha = 0; // Index Head
21    long long pos_beta = tambang.size - 1; // Index Tail
22    // Main Loop / Process (input loncatan dan singular
23    event)
24    for (int i = 0; i < R; i++) {
25        // Max 10^18 tidak bisa pake int tapi pake long long
26        / int64
27        long long X, Y;
28        cin >> X >> Y;

```

```

29      // Cek Jika N == 0 (semua karakter sudah hancur)
30  hentikan loop
31      if (tambang.size == 0)
32          break;
33      // Optimasi Lompatan
34      pos_alpha = (pos_alpha + X) % tambang.size;
35      long long langkah_beta = Y % tambang.size;
36      pos_beta = (pos_beta - langkah_beta + tambang.size) %
37  tambang.size;
38      char isi_alpha = tambang.get(pos_alpha);
39      char isi_beta = tambang.get(pos_beta);
40      // Cek Tabrakan atau Resonansi
41      if (pos_alpha == pos_beta) {
42          // panggil fungsi dari DoubleLinkedList
43          tambang.delete_idx(pos_alpha);
44          // Pastikan tidak melakukan Modulo 0 jika semua
45  karakter baru saja hancur
46          if (tambang.size > 0) {
47              // Alpha tetap di posisinya secara angka, tapi
48  karena size berkurang,
49              // pastikan dia tidak out of bounds.
50              pos_alpha = pos_alpha % tambang.size;
51              // Beta mundur 1. Gunakan + size lalu Modulo agar
52  aman dari angka
53              // negatif
54              pos_beta = (pos_beta - 1 + tambang.size) %
55  tambang.size;
56          }
57          // PROBLEM Awalnya saya ubah menjadi circular
58  (pos_alpha + 1) %
59          // tambang.size == pos_beta || (pos_beta + 1) %
60  tambang.size == pos_alpha
61          // tetepi karena tidak benar semua saya coba kembali
62  pake yang biasa eh
63          // malah 100
64      } else if (pos_alpha + 1 == pos_beta || pos_beta + 1
65  == pos_alpha) {
66          // Resonansi
67          // Tukar karakternya menggunakan set()
68          tambang.set(isi_beta, pos_alpha);
69          tambang.set(isi_alpha, pos_beta);
70      }
71  }
72  tambang.display();

```


73	return 0;
74	}

Table 6. Source Code prob_b.cpp

B. Output Program

```

kimsora .../linked-list ? main [?] v15.2.022s
> ./prob_b
4 2
A B C D
1 1
2 3
ACB

kimsora .../linked-list ? main [?] v15.2.08s
> ./prob_b
5 3
V W X Y Z
100000 100000
2 2
5 5
VWYZ

```

Gambar 2. Screenshot Hasil Jawaban Soal 2

C. Pembahasan

File ini adalah main file dimana program utama berjalan.

Alur Algoritma

- Inisialisasi data: Program membuat struktur baru dan memasukan karakter awal sebanyak N menggunakan add_back.
- Pelacakan posisi: Program ini membuat posisi alpha dan beta kedalam bentuk variable numerik sebagai kordinatnya yaitu pos_alpha=0 (head) dan pos_beta = size – 1 (tail).
- Perulangan Ronde: menjalankan perulangan ronde sebanyak R. pada perulangan ini lah proses utamanya.
- Pergerakan alpha dan beta: lompata X dan Y dihitung dengan optimasi modulo, alpha maju dengan menambahkan sisa bagi jaraknya $pos_alpha = (pos_alpha + X) \% size$.

Beta mundur menggunakan pengurangan yang dilindungi dengan nilai dasar size supaya aman dari nilai negatif $\text{pos_beta} = (\text{pos_beta} - Y \% \text{size} + \text{size}) \% \text{size}$.

- Event Tabrakan atau Resonansi:
 - jika $\text{pos_alpha} == \text{pos_beta}$ (Tabrakan): Program akan memanggil fungsi `delete_idx(pos_alpha)`. Jika list masih belum kosong posisi alpha dan beta akan otomatis terkalkulasi ulang terhadap ukuran list yang baru saja berkurang.
 - Jika tidak bertabrakan program akan cek resonansi $((\text{pos_alpha} + 1) == \text{pos_beta})$: Program akan menukar karakter dua node menggunakan `set()`.
- Output Tampilan: Setelah semua R ronde selesai list karakter langsung dicetak dari head hingga tail dengan fungsi `display()`.

Kenapa Ini Bekerja?

Algoritma ini aman karena adanya pemisahan arsitektur. Program tidak perlu melakukan pengaturan jalinan pointer list saat terjadinya tabrakan, program hanya perlu memanggil `delete_idx` yang sudah disiapkan. Fungsi yang sudah kita buat di soal sebelumnya ini akan mengubah ikatan `prev` dan `next` secara otomatis. Dengan cara ini program cukup menkalibrasi ulang `pos_alpha` dan `beta` terhadap `size` baru saja.

Mekanisme kalibrasi setelah tabrakan dan `delete_idx`, node akan bergeser satu langkah ke kiri. Karena itu posisi yang sebelumnya punya node yang dihapus sekarang sudah ditempati oleh node kanan nya. `pos_alpha` tidak perlu di ubah manual, cukup cek dengan $\text{pos_alpha} \% \text{size}$ supaya tidak melebihi batas.

Algoritma ini juga menangani kondisi dimensi kosong pada loop utama nya sesuai dengan instruksi dari soal, terdapat pengecekan `if (timbang → size == 0) break`; ini adalah hal yang krusial jika node sudah kosong dan program tetap berjalan ke perhitungan $X \% \text{timbang.size}$, maka akan terjadi pembagian nol, yang akan membuat crash.

Analisis Kompleksitas

Kompleksitas Waktu: $O(r \cdot n)$. Karena ada loop utama(ronde) sebanyak R , dan dilamnya terdapat beberapa fungsi yang memiliki kompleksitas waktu $O(n)$, dengan demikian disetiap ronde program menghabiskan rata-rata waktu $O(n)$. Maka total nya $O(r \cdot n)$. kenapa disini tidak $O(n^2)$? Karena R disini merupakan para meter yang berbeda dari N , ya jika nilai mereka sama akan tetap $O(n^2)$ tetapi dalam kasus ini R nya beda. Jadi ini hampir sama dengan $O(n^2)$ tetapi tidak konsisten.

Kompleksitas Ruang: $O(n)$ karena algoritma menyimpan N karakter awal kedalam struktur data linked list.

TAUTAN GIT

Repo lengkap : <https://github.com/biahlil/Algoritma-dan-Struktur-Data.git>
Linked List : <https://github.com/DSA26-ULM/module-3-linked-list-biahlil-1.git>